# Projekt - Phase 02

## Plattform zum Teilen und Bewerten von Filminformationen

5430UE Web and Data Engineering

10. Juni 2026

Prof. Dr. Michael Granitzer, Tobias Fuchs

# Aufgabenstellung

# Aufgabenstellung

Das Ziel des Gesamtprojekts bleibt die Entwicklung einer webbasierten Plattform zur Darstellung, Suche und Bewertung von Filminformationen. Nachdem in Phase 01 das Design und das Frontend entstanden sind, folgt nun in **Phase 02** die technische Implementierung des Backends und die vollständige Integration des bestehenden Frontends.

# Aufgabenstellung

## Funktionale Anforderungen (identisch zu Phase 01) (1/3)

- **Filme anlegen:** Nutzer sollen über ein Formular neue Filme anlegen können. Ein Eintrag muss dabei Informationen wie Titel, Erscheinungsjahr, Beschreibung/Kurzinhalt, Dauer, Genre, Altersfreigabe, eine Liste von Schauspielern sowie ein Filmbild enthalten.
- **Übersichtsseite:** Die Startseite zeigt die zuletzt hinzugefügten Einträge an (z.B. die letzten 20). Jeder Eintrag wird hierbei mit Titel, Genre und Erstellungsdatum dargestellt. Je nach Gestaltung kann auch das Filmbild angezeigt werden (z.B. als Kacheldarstellung).
- **Bewertungsfunktion:** Nutzer können Filme mit 1 bis 5 Sternen bewerten. Dabei soll die durchschnittliche Bewertung pro Film (z.B. 3,5 von 5 Sternen) angezeigt werden. Zudem soll sichergestellt werden, dass Benutzer einen Film nur einmal bewerten können.

# Aufgabenstellung

## Funktionale Anforderungen (identisch zu Phase 01) (2/3)

- **Top-Bewertungen:** Auf der Startseite sollen die aktuell am besten bewerteten Filme (Top 3 oder Top 5) hervorgehoben angezeigt werden.
- **Suche und Filterung:** Die Filme sollen nach Schlüsselworten, etwa via Titel oder Schauspieler, durchsucht werden können, und das Suchergebnis wird angezeigt. Zudem können Benutzer die angezeigten Filme nach Genre, Altersfreigabe und Bewertung filtern.
- **Detailseite:** Von jedem angezeigten Film auf der Übersichtsseite soll jederzeit eine separate Detailseite aufgerufen werden können. Diese zeigt die vollständigen erfassten Informationen zum Film, wie die Kurzbeschreibung, die Liste der Schauspieler, etc.

# Aufgabenstellung

## Funktionale Anforderungen (identisch zu Phase 01) (3/3)

- **Kommentarfunktion:** Benutzer können zu Filmen Kommentare hinterlassen, die mit Autor, ggf. Titel und Inhalt angezeigt werden. (Optional: Nachträgliche Bearbeitung durch den Autor ermöglichen).
- **Validierung:** Eine Validierung der Eingaben erfolgt bereits clientseitig mittels JavaScript.
- **Authentifizierung:** Es ist kein Authentifizierungssystem zu implementieren.

# Aufgabenstellung

## Zusätzliche Informationen und Spezifizierungen

Um die Benutzererfahrung zu optimieren und die Robustheit der Anwendung zu gewährleisten, sind folgende Erweiterungen im Frontend zwingend umzusetzen:

- **Access/Navigation Management:** Auf der Detailseite eines Films soll den Nutzern visuell signalisiert werden, ob sie für diesen Eintrag bereits eine Bewertung abgegeben haben. Dies kann beispielsweise durch das Ausgrauen des Bewertungs-Buttons oder eine entsprechende Anpassung der Sterne-Darstellung erfolgen. (Optional: Analog für Kommentare und deren Bearbeitbarkeit)
- **Erweiterte Client-Validierung:** Fehleingaben beim Anlegen eines Films oder Verfassen eines Kommentars müssen clientseitig abgefangen werden, bevor ein Request an das Backend gesendet wird. Dem Benutzer ist ein klar verständlicher, visueller Hinweis zu geben, wie die Eingabe korrigiert werden kann (z. B. Überprüfung auf minimale/maximale Textlängen, korrekte Zahlenformate bei der Filmdauer oder Pflichtfeld-Prüfungen).
- **Umgang mit Server-Validierungsfehlern:** Schlägt eine Validierung im Backend fehl (z. B. weil ein Filmtitel bereits existiert), darf die Anwendung nicht abstürzen. Das Frontend muss diese serverseitigen Fehlermeldungen abfangen und dem Nutzer dynamisch (z. B. über rote Text-Benachrichtigungen,...) anzeigen.

# Architekturprinzipien und technische Anforderungen

# Architekturprinzipien und technische Anforderungen

Das Backend muss zwingend auf **Java und Spring Boot** basieren. Das Projekt ist als Build-Management-kompatibles Projekt mittels **Maven** aufzusetzen.

# Schichtenarchitektur (Separation of Concerns)

Implementieren Sie das Backend strikt unter Beachtung des Separation of Concerns-Prinzips. Das Programm ist durch eine klare Aufteilung in folgende Schichten modularisiert aufzubauen:

- **Controller-Schicht:** Bereitstellung der REST-Endpunkte, Entgegennahme von HTTP-Requests, Validierung der eingehenden Daten und Steuerung der HTTP-Schnittstelle.
- **Service-Schicht (Geschäftslogik):** Kapselung der eigentlichen Anwendungslogik, Durchführung komplexerer Berechnungen (z. B. Durchschnittsbewertungen) und Koordination zwischen Repositories und Controllern.
- **Datentransferobjekte (DTOs):** Entkopplung der internen Datenbank-Entitäten von der öffentlichen API-Schnittstelle. Daten sollten vor dem Senden an den Client in DTOs transformiert werden.
- **Persistenzschicht (Entities & Repositories):** Definition des relationalen Datenmodells (Entities) und Abwicklung aller Datenbankzugriffe über Spring Data JPA Repositories.

Die Einhaltung dieser Schichtenarchitektur ist eine grundlegende Bewertungsanforderung.

# RESTful API-Design

Entwickeln Sie eine konsistente RESTful API zur Kommunikation zwischen Frontend und Backend. Die API stellt Endpunkte bereit, um neue Filme hinzuzufügen, vorhandene Einträge zu durchsuchen, zu filtern und Bewertungen sowie Kommentare zu übermitteln. Die clientseitigen `fetch()`-Aufrufe aus Phase 01 sind auf diese Endpunkte umzustellen. Analog zur Client-Validierung sollen sinnvoll serverseitig Validierungsfehler abgefangen und als verständliche Rückmeldungen an das Frontend übergeben werden.

# Datenbank & Initialisierung

Als Datenbank wird standardmäßig eine H2-In-Memory-Datenbank verwendet, um ein unkompliziertes Setup ohne externe Abhängigkeiten zu ermöglichen. Alternativ ist nach Absprache in begründeten Ausnahmefällen die Nutzung einer lokalen PostgreSQL-Datenbank zulässig.

Beim Start der Anwendung soll die Datenbank automatisch mit einigen vordefinierten Beispieleinträgen (Filme, Kommentare, Bewertungen) befüllt werden. Dies kann z. B. über eine `data.sql`-Datei im Ressourcen-Ordner realisiert werden.

*Optional:* Um Flexibilität zu gewährleisten, sollte dieser Initialisierungsvorgang über ein Property in der `application.properties` (z. B. `app.db.init=true`) optional an- und ausschaltbar sein.

# Sicherheit (Einfacher Missbrauchsschutz)

Die API soll ohne Authentifizierungssystem öffentlich zugänglich sein. Dennoch müssen grundlegende Sicherheitsmechanismen implementiert werden, um zu verhindern, dass ein Nutzer mehrfach für denselben Eintrag abstimmt oder die Kommentarfunktion missbraucht. Dies ist über einfache Mechanismen wie Cookies oder IP-basierte Beschränkungen zu lösen. Dem Nutzer soll auf der Detailseite signalisiert werden, wenn er bereits eine Bewertung abgegeben hat bzw. nur die Bearbeitung der eigenen Kommentare ermöglicht werden.

# Cookie- & IP-Check: Minimalbeispiel

**Allgemeines Minimalbeispiel zur Umsetzung:**

Das folgende Beispiel zeigt schematisch, wie HTTP-Requests auf Basis von Cookies und der IP-Adresse validiert werden können. Es dient als konzeptionelle Vorlage, die Sie auf Ihren Anwendungsfall adaptieren müssen (siehe auch das Minimal Spring Boot Projekt *demoCookieIP*).

```
@PostMapping("/vote/{itemId}")
public ResponseEntity<String> submitVote(@PathVariable Long itemId, HttpServletRequest request, @CookieValue(value = "voted_items", defaul
  // 1. IP-Adresse auslesen
  String clientIp = request.getRemoteAddr();

  String ipVoteKey = clientIp + "_" + itemId;

  // 2. Schutz-Pruefung (Cookie ODER IP)
  boolean hasCookie = votedCookie.contains("[" + itemId + "]");
  boolean hasIpVoted = voteDatabase.contains(ipVoteKey);

  // Falls bereits mindestens eine der Sperrbedingungen zutrifft, wird eine
  // entsprechende Meldung gebaut und der Response mit StatusCode und einem
  // Fehlertext zurueckgegeben
  if (hasCookie || hasIpVoted) {
    String grund = hasCookie ? "Cookie-Erkennung" : "IP-Erkennung (" + clientIp + ")";
    return ResponseEntity.status(HttpStatus.FORBIDDEN)
    .body("Gesperrt via " + grund + "! Sie haben fuer Item " + itemId + " bereits abgestimmt.");
```

# Cookie- & IP-Check: Lokales Testen

**Hinweise zum lokalen Testen:**

- Rufen Sie die Demo-Anwendung explizit über die IPv4-Adresse 127.0.0.1 statt über localhost auf. Dieses Verhalten, dass die IP-Sperre manchmal nicht funktioniert, resultiert daraus, dass Webbrowser beim Aufruf von localhost wechselweise die IPv4-Adresse (127.0.0.1) oder die IPv6-Adresse (::1) auflösen; da der Server in der einfachen Demo diese als zwei unterschiedliche IP-Adressen interpretiert, wird die IP-basierte Sperre umgangen.
- **Cookie-Check:** Nutzen Sie den Inkognito-Modus Ihres Browsers oder löschen Sie die Cookies in den Entwicklertools (F12 $\rightarrow$ Application $\rightarrow$ Cookies), um eine erneute Stimmabgabe zu erzwingen.
- **IP-Check:** Wenn Sie lokal testen, gibt `request.getRemoteAddr()` in der Regel `127.0.0.1` (IPv4) oder `0:0:0:0:0:0:0:1` (IPv6 localhost) zurück.

# Allgemeine Vorgaben

# Allgemeine Vorgaben

- **Feature-basierte Implementierung:** Genau wie in Phase 01 gilt, dass Front- und Backend-Komponenten eines Features als logische Einheit implementiert werden müssen. Das bedeutet, dass eine Funktionalität (wie z. B. die Suche oder das Bewertungssystem) als logische Einheit betrachtet wird und sowohl im Frontend als auch im Backend im Zusammenhang von der gleichen Person implementiert wird. Ausnahmen sind unter vorheriger Absprache mit dem Übungsleiter zulässig

# Allgemeine Vorgaben

## a) FimGit und GitLab Issues

- Nutzen Sie weiterhin ausschließlich das für Ihr Team erstellte FimGit-Repository.
- Nutzen Sie weiterhin auch das Feature GitLab Issues zur Aufgabenplanung, Zuweisung und um einen steten Überblick zu haben, was noch zu tun ist. Die lückenlose Nutzung geht in die Bewertung mit ein. Nutzen Sie ein für Ihr Team sinnvolles Git-Branching-Modell (z. B. GitFlow).
- Hinweis: Zugang zum FIM Gitlab erfordert einen FIM-Account. Weitere Informationen zum Account-Management des FIM-Accounts finden Sie auf der FIM-Webseite. Bei Bedarf können Sie EduVPN nutzen, um sich in das Universitätsnetzwerk einzuloggen.

# Allgemeine Vorgaben

## b) Projektstruktur und Technologien

Das in Phase 01 erstellte Frontend wird vollständig in das Maven-basierte Spring Boot-Projekt integriert.

**Zusammenfassung des Technologiestacks für Phase 02:**

- **Frontend:** HTML5, CSS (Frameworks Bootstrap ist erlaubt), Vanilla JavaScript (Fetch API).
- **Backend:** Java (JDK 17 oder höher), Spring Boot (Spring MVC für REST, Spring Data JPA).
- **Build-Tool & Abhängigkeiten:** Maven (`pom.xml`)
- **Datenbank:** H2-Datenbank (In-Memory) (Begründete Fälle: lokal installierte PostgreSQL-Instanz.)

# Allgemeine Vorgaben

## Vorgegebene Projektstruktur Phase 02:

```
/spring-projekt-root
  |-- pom.xml                              (Maven Konfiguration und Dependencies)
  |-- src/
    |-- main/
      |-- java/de/uni/passau/wde/      (WDE-Backend-Code-Wurzelverzeichnis)
        |-- controller/              (REST-Endpunkte)
        |-- dto/                     (Datentransferobjekte)
        |-- entity/                  (JPA-Datenbankmodelle)
        |-- repository/              (Spring Data Repositories)
        |-- service/                 (Geschaeftslogik)
    |-- resources/
      |-- static/                  (INTEGRATION DES FRONTENDS AUS PHASE 01)
        |-- html/                (Unterseiten)
        |-- css/                 (Stylesheets)
        |-- assets/              (Bilder, Logos, etc.)
        |-- js/                  (JS-Dateien angepasst auf REST-API)
        |-- index.html           (Hauptdatei der Anwendung auf localhost:8080)
    |-- templates/               (Bleibt bei reinen REST-Anwendungen leer)
    |-- application.properties   (Konfiguration für Datenbank & JPA)
```



# Allgemeine Vorgaben

## c) Dokumentation und Setup Guide

**Abgabedokument:**

- Kurze Übersicht der Features und der jeweils allein zuständigen Person gemäß des feature-basierten Ansatzes.
- Anleitung, wie die Anwendung zu starten ist, ggf. welche Vorbereitungen im Vorfeld getroffen werden müssen (Testsystem des Korrektors: Windows mit IntelliJ-IDE, Java JDK installiert, Internetverbindung vorhanden, ggf. lokale PostgresDB vorhanden).
- Optional: Implementierte Besonderheiten, ungelöste Probleme.

# Allgemeine Vorgaben

**Code-Dokumentation:**

- Dokumentieren Sie Ihren Code sinnvoll und knapp, aber ausreichend (Keine Romane, kein KI-generierter Spam).
- Verpflichtend: Verwendung des `@author`-Tags in JavaDocs bei sämtlichen Java-Klassen und JavaScript-Dateien. Bei CSS- und HTML-Dateien reicht eine entsprechende Kennzeichnung über Inline-Kommentare.

# Allgemeine Vorgaben

**Setup Guide (README.md):**

Ergänzen Sie Ihr Repository um eine prägnante, gut strukturierte `README.md`-Datei auf der obersten Ebene.

Dokumentieren Sie darin verständlich die folgenden Kernpunkte:

1. **Voraussetzungen:** Benötigte Java-Version, Build-Tools, etc.
2. **Einrichtung und Start:** Befehle zum Bauen und Ausführen (z. B. `./mvnw spring-boot:run`).
3. **Datenbank-Konfiguration:** Zugriff auf die H2-Console (Pfad, JDBC-URL) bzw. Einstellungen für PostgreSQL sowie Steuerung der Beispieldaten-Initialisierung.
4. **API-Übersicht:** Eine kurze tabellarische Aufstellung der wichtigsten REST-Endpunkte mit HTTP-Methode, Pfad und einer kurzen Funktionsbeschreibung.
5. **Kurzanleitung:** Ein kurze Anleitung zur Bedienung der Funktionen der Anwendung.

# Abgaben und Vorstellung

# Abgaben und Vorstellung

**Abgabe:**

- Eine verbindliche Abgabe in beiden Phasen.
- Abgabe für Phase 02 bis zum **24.06.2026 um 23:59 Uhr MESZ**.
- Einsendung per E-Mail mit dem Betreff „WDE [GruppenNr]“. Im Nachrichtentext zusätzlich Namen und Matrikelnummern der Teilnehmenden, einen Link zum FIMGit-Repository und den Hash des Commits der Abgabe angeben.
- Bestandteile der Abgabe:

  1. Abgabedokument via PDF
  2. Projekt als ZIP-Datei
  3. Vollständige `README.md` im Git-Repository

  *Hinweis:* Gehen Sie hierzu analog zu Phase 01 vor.

# Abgaben und Vorstellung

**Präsentation Phase 02:**

- Kurze Präsentation und Demonstration Ihres funktionsfähigen Projekts in den Wochen vom 29.06. bis 10.07.2026. Termine sind nach Projektabgabe via Stud.IP buchbar.
- Im Anschluss an die Live-Demonstration im Browser folgt ein kurzes Fachgespräch. Hierbei werden gezielte Fragen zu Ihren Architektur- und Implementierungsentscheidungen gestellt, die sich insbesondere auf die von den jeweiligen Teammitgliedern individuell verantworteten Features beziehen (Überprüfung der Eigenständigkeit).

# Weiteres

# Weiteres

- Nutzung einer Plagiatssoftware zur Sicherstellung der Originalität des Codes. Keine Verwendung von Code aus früheren Projekten oder Online-Quellen.
- Generative Modelle wie ChatGPT dürfen zur Unterstützung des Projekts verwendet werden, eine Dokumentation in welcher Form ist jedoch notwendig. Die inhaltliche Verantwortung verbleibt vollständig bei den Teammitgliedern.

# Materialien

# Demoprojekt zu Cookies und IP-Adresse

Projekt zur Demonstration des Cookie- und IP-Checks in Spring Boot (Minimal Spring Boot Projekt *demoCookieIP*):
demoCookieIP.zip